

Kubernetes and Cloud Native Associate - KCNA

Scheduling

Static Pods

In this lesson, we explore the concept of static pods in Kubernetes and explain how they differ from standard pods managed by the control plane. Previously, we reviewed the Kubernetes architecture and the role of the kubelet, which typically communicates with the Kubernetes API server to receive pod specifications. These instructions are generated by the kube scheduler and stored in etcd. But what occurs when there is no API server, scheduler, controllers, or even an etcd cluster—that is, when the kubelet is operating on a standalone node?

Imagine you have a server with only the kubelet and Docker installed. In this scenario, there is no API server to provide pod details. Since a pod requires a pod definition file to be created, you can configure the kubelet to read these definitions directly from a designated directory on the host. The kubelet continuously monitors this directory, creates pods based on the files it finds, restarts them if they crash, and deletes them when the corresponding files are removed. Pods created using this method are referred to as static pods.



Note

Static pods support only pod objects. More advanced Kubernetes objects such as ReplicaSets, Deployments, or Services require the full control plane and cannot be defined using the static pod mechanism.

Configuring the Static Pod Directory

The directory containing static pod definitions can be any folder on the host. Its location is provided to the kubelet using the pod manifest path option when starting the service. For example, in a systemd service file, you might see:

```
ExecStart=/usr/local/bin/kubelet \  
  --container-runtime=remote \  
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \  
  --pod-manifest-path=/etc/Kubernetes/manifests \  
  --kubeconfig=/var/lib/kubelet/kubeconfig \  
  --network-plugin=cni \  
  --register-node=true \  
  -v=2
```

Alternatively, you can specify the pod manifest path in an external configuration file. In clusters set up with tools like kubeadm, the kubelet is often configured with a flag that points to a config file. For example:

```
ExecStart=/usr/local/bin/kubelet \  
  --container-runtime=remote \  
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \  
  --config=kubeconfig.yaml \  
  --kubeconfig=/var/lib/kubelet/kubeconfig \  
  --network-plugin=cni \  
  --register-node=true \  
  -v=2
```

The configuration file (e.g., kubeconfig.yaml) would include:

```
staticPodPath: /etc/kubernetes/manifests
```

When inspecting an existing cluster, check whether the pod manifest path option is set directly in the service file or referenced in a configuration file. This will indicate where the static pod definition files are stored. Once the pod definitions are placed

in the designated directory, the kubelet automatically creates the pods. Since these pods are created without the API server, they do not appear in typical `kubectl` output. Instead, you can verify the running containers using Docker:

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
8e5d4c4db7b6	busybox	"sh -c 'echo Hello K..."	20 seconds ago
f6737e1149cb	k8s.gcr.io/pause:3.1	"/pause"	24 seconds ago

Static Pods in a Cluster Environment

When a node is part of a cluster with a functioning API server, the kubelet can create pods from two sources simultaneously:

1. Pod definition files located in the static pods folder.
2. Pod specifications provided via the HTTP API endpoint by the Kubernetes API server.

In such cases, when you run the command on the master node:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
static-web-node01	0/1	ContainerCreating	0	29s

static pods are listed alongside other pods because the kubelet creates a mirror pod object for each static pod in the API server. These mirror pods are read-only representations and cannot be modified or deleted via the API server. To remove or update a static pod, you must modify the corresponding pod definition file in the static pod manifest directory. Additionally, note that the pod name is automatically appended with the node name (e.g., "node01") to help distinguish its origin.

Static pods are especially useful for deploying Kubernetes control plane components. For instance, you can install the kubelet on all master nodes and create pod definition files for essential components such as the API server, controller-

manager, and etcd. These definitions are stored in the designated manifest folder, and the kubelet manages and restarts the services automatically. This approach simplifies the setup by eliminating the need to manually manage binaries or services. In clusters set up with the kubeadm tool, you can see control plane components running as pods in the kube-system namespace:

```
kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-78fcd6894-hwrq9	1/1	Running	0	16m
coredns-78fcd6894-rzhjr	1/1	Running	0	16m
etcd-master	1/1	Running	0	15m
kube-apiserver-master	1/1	Running	0	15m
kube-controller-manager-master	1/1	Running	0	15m
kube-proxy-lzt6f	1/1	Running	0	16m
kube-proxy-zm5qd	1/1	Running	0	16m
kube-scheduler-master	1/1	Running	0	15m
weave-net-29z42	2/2	Running	1	16m
weave-net-snm1l	2/2	Running	1	16m

Static Pods vs. DaemonSets

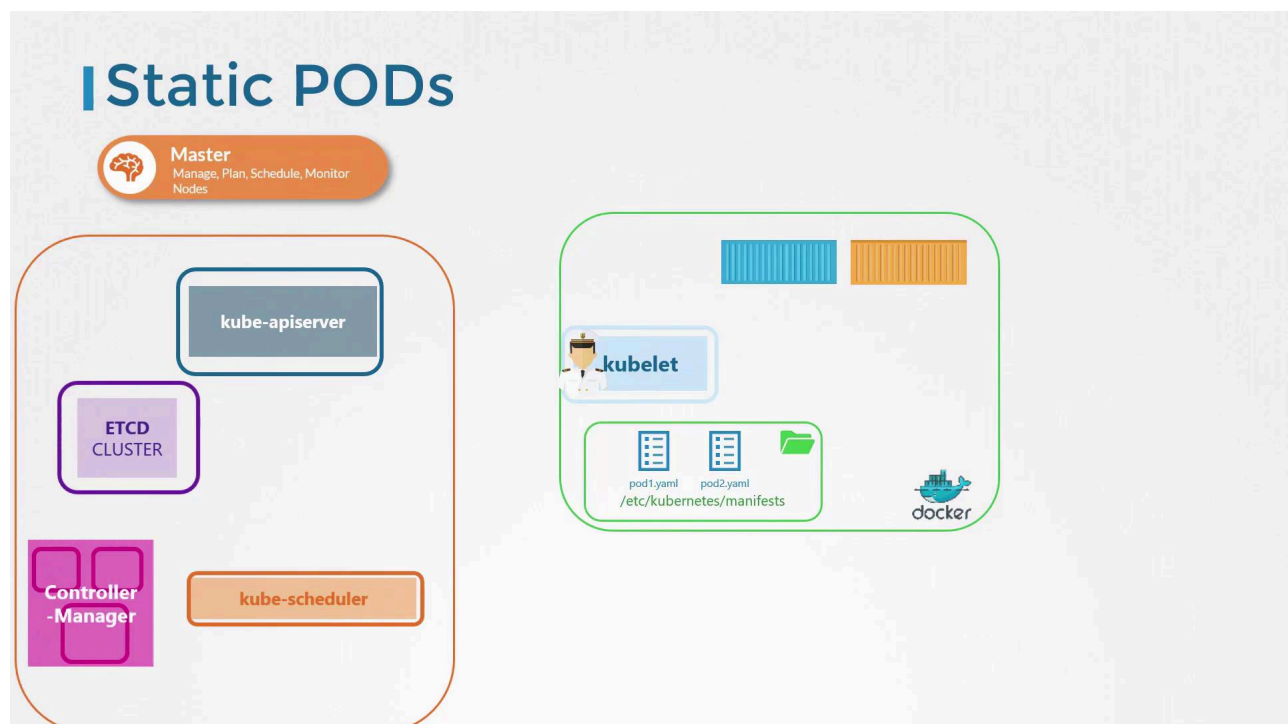
A common question is how static pods compare with DaemonSets. DaemonSets ensure that a specific pod runs on all nodes in a cluster, managed by a DaemonSet controller via the API server. In contrast, static pods are created directly by the kubelet without intervention from the API server or other control plane components. Both methods bypass the kube scheduler.

Static PODs vs DaemonSets

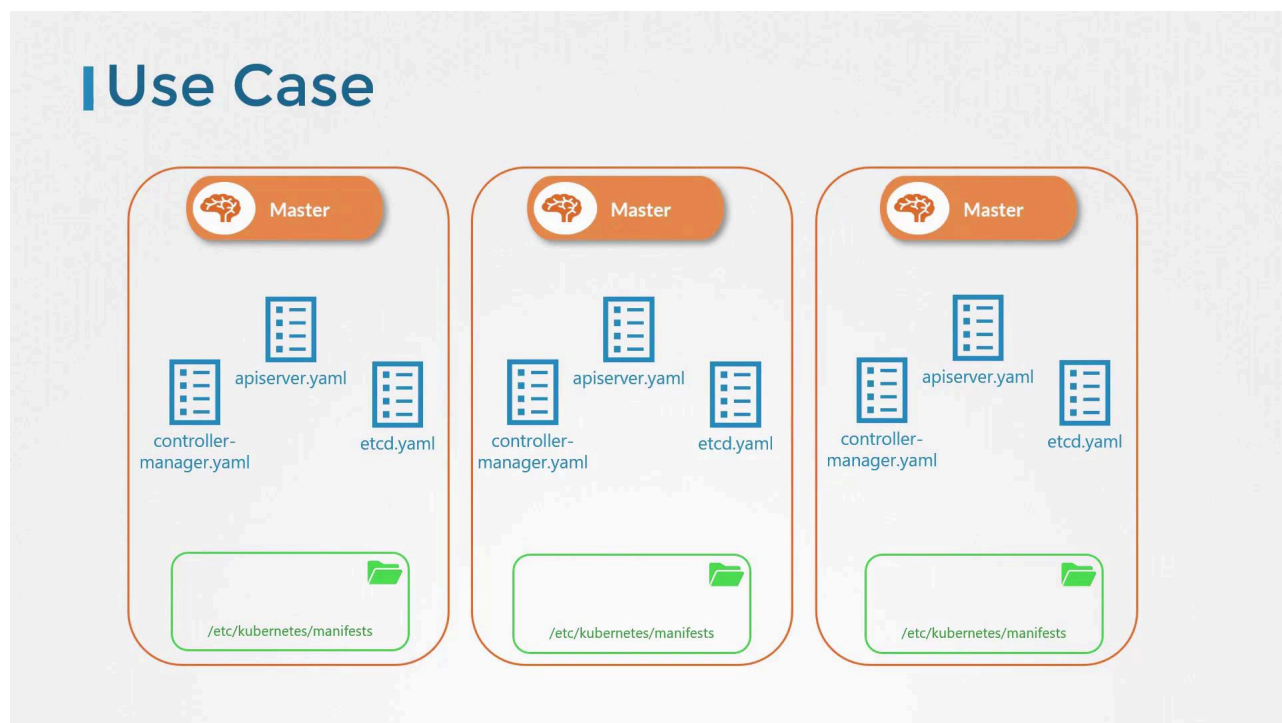
Static PODs	DaemonSets
Created by the Kubelet	Created by Kube-API server (DaemonSet Controller)
Deploy Control Plane components as Static Pods	Deploy Monitoring Agents, Logging Agents on nodes
Ignored by the Kube-Scheduler	

Static Pods Architecture

Static pods play a critical role in deploying control plane components. The following architectural diagram shows how static pod definitions stored in a designated folder are used by the kubelet to deploy essential components:



Another diagram below presents a typical control plane deployment across multiple master nodes using static pods stored in `/etc/kubernetes/manifests`:



Summary

Static pods offer a powerful mechanism for running pods independently of the Kubernetes control plane. Managed directly by the kubelet, they are continuously monitored based on pod definition files placed in a designated directory. While static pods do not benefit from advanced features such as ReplicaSets or Deployments, they are essential for bootstrapping and maintaining control plane components, especially during initial cluster setup.



Tip

Always update or remove static pods by modifying the corresponding pod definition file in the manifest directory, as any changes made through the API server will not affect static pods.

By experimenting with static pods in both standalone and clustered environments, you can build confidence in managing essential Kubernetes services and fine-tuning your cluster's configuration. For more insights on Kubernetes architecture and management, explore resources like [Kubernetes Documentation](#) and [Kubernetes Basics](#).



Watch Video

Watch video content

Previous

◀ [DaemonSets](#)

Next

[Multiple Schedulers](#) ▶



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▼